

Workshop: Practical Function Programming

Concepts of Programming Languages

Part 1: OCaml AUA

Part 2: Practice Problems

Practice Problem

Implement the function

val negatives : int list -> int list

*so that **negatives l** is the list negative numbers appearing in **l**. Also implement the function*

val gets : 'a -> ('a * 'b) list -> 'b list

*so that **gets key l** is the list of values **v** such that **(key, v)** is a member of **l***

Write a single function that can be used to implement both

Practice Problem

*Implement **filter** and **append (@)** using **List.fold_right***

Practice Problem

```
let rec insert le v l =  
  match l with  
  | [] -> [v]  
  | x :: xs ->  
    if le v x  
    then v :: l  
    else x :: insert le v l
```

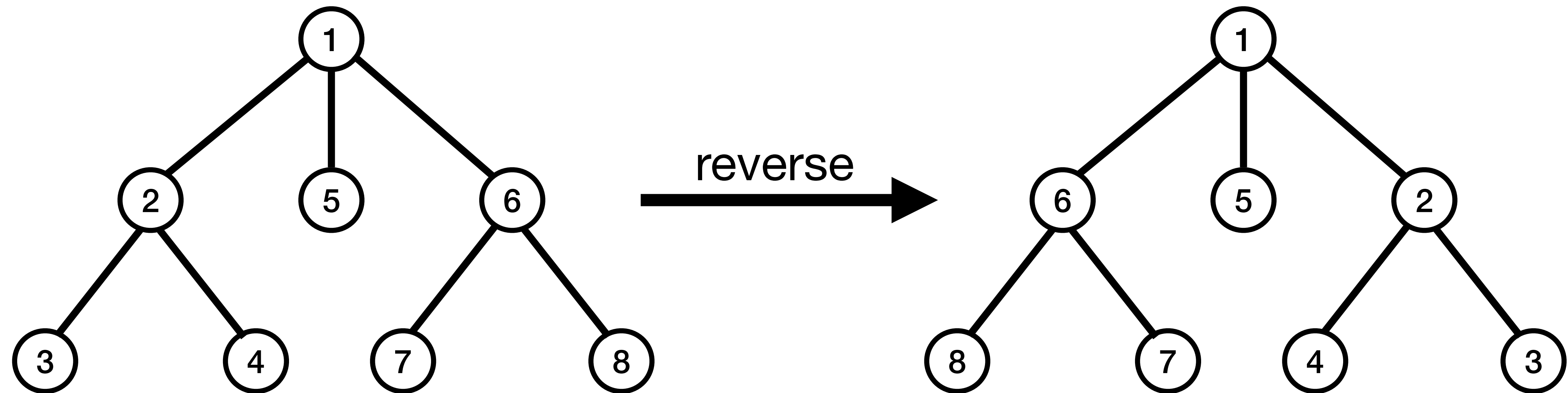
In terms of **fold_left** and **insert** implement

val sort : ('a -> 'a -> bool) -> 'a list -> 'a list

so that **sort le l** is **l** in sorted order **le**

Practice Problem

*Implement **reverse** for binary trees*



Part 3: Practical Stuff

Modular Programming

High Level

High Level

Modules capture multiple programming patterns with a single construct:

High Level

Modules capture multiple programming patterns with a single construct:

- 1. Namespaces:** separate coding into logical units

```
module Interpreter = struct
  let type_check = ...
  let eval = ...
end
```

(1)

High Level

Modules capture multiple programming patterns with a single construct:

- 1. Namespaces:** separate coding into logical units
- 2. Abstraction/Encapsulation:** abstracts away implementation details and organizing core functionality

```
module Interpreter = struct
  let type_check = ...
  let eval = ...
end
```

(1)

```
module Stack = struct
  type 'a t = 'a list
  let push x s = x :: s
  let pop s = match s with
    | [] -> None
    | x :: xs -> Some (x, xs)
end
```

(2)

High Level

Modules capture multiple programming patterns with a single construct:

- 1. Namespaces:** separate coding into logical units
- 2. Abstraction/Encapsulation:** abstracts away implementation details and organizing core functionality
- 3. Code Reuse:** general code that can be instantiated in different settings

```
module Interpreter = struct
  let type_check = ...
  let eval = ...
end
```

(1)

```
module Stack = struct
  type 'a t = 'a list
  let push x s = x :: s
  let pop s = match s with
    | [] -> None
    | x :: xs -> Some (x, xs)
end
```

(2)

```
module VarSet = Set.Make(String)
module Context = Map.Make(String)
```

(3)

Structures

```
module Foo = struct
  let double (x : int) : int = x + x

  let is_whitespace (c : char) =
    List.mem c [' '; '\n'; '\t'; '\r']

  let version = 225

  exception MyException
end
```

Structures

A **structure** is a collection
of *let-definitions*

```
module Foo = struct
  let double (x : int) : int = x + x

  let is_whitespace (c : char) =
    List.mem c [' '; '\n'; '\t'; '\r']

  let version = 225

  exception MyException
end
```

Structures

A **structure** is a collection of *let-definitions*

Structures aren't first-class, we *must* use the **module** keyword when defining a structure

```
module Foo = struct
  let double (x : int) : int = x + x

  let is_whitespace (c : char) =
    List.mem c [' '; '\n'; '\t'; '\r']

  let version = 225

  exception MyException
end
```

Structures

A **structure** is a collection of *let-definitions*

Structures aren't first-class, we *must* use the **module** keyword when defining a structure

We can put anything in a structure that we can put in a standalone .ml file

```
module Foo = struct
  let double (x : int) : int = x + x

  let is_whitespace (c : char) =
    List.mem c [' '; '\n'; '\t'; '\r']

  let version = 225

  exception MyException
end
```

Signatures

```
module type F00 =  
  sig  
    val double : int -> int  
    val is_whitespace : char -> bool  
    val version : int  
    exception MyException  
  end
```

Signatures

A **signature** is a collection of *specifications*

```
module type F00 =  
  sig  
    val double : int -> int  
    val is_whitespace : char -> bool  
    val version : int  
    exception MyException  
  end
```

Signatures

A **signature** is a collection of *specifications*

A specification is a name together with a type

```
module type F00 =  
  sig  
    val double : int -> int  
    val is_whitespace : char -> bool  
    val version : int  
    exception MyException  
  end
```

Signatures

A **signature** is a collection of *specifications*

A specification is a name together with a type

signatures are **module types**

```
module type F00 =  
  sig  
    val double : int -> int  
    val is_whitespace : char -> bool  
    val version : int  
    exception MyException  
  end
```

Signatures

A **signature** is a collection of *specifications*

A specification is a name together with a type

signatures are **module types**

A module **implements** a signature if it's defined as a structure which has the values required by the signature

```
module type F00 =  
  sig  
    val double : int -> int  
    val is_whitespace : char -> bool  
    val version : int  
    exception MyException  
  end
```

Signature Inference and Interface Files

```
let double (x : int) : int = x + x

let is_whitespace (c : char) =
  List.mem c [' '; '\n'; '\t'; '\r']

let version = 225

exception MyException
```

foo.ml

```
val double : int -> int

val is_whitespace : char -> bool

val version : int

exception MyException
```

foo.mli

Signature Inference and Interface Files

```
let double (x : int) : int = x + x

let is_whitespace (c : char) =
  List.mem c [' '; '\n'; '\t'; '\r']

let version = 225

exception MyException
```

foo.ml

```
val double : int -> int

val is_whitespace : char -> bool

val version : int

exception MyException
```

foo.mli

In most cases, OCaml infers the signature of a given module (the annotation is optional)

Signature Inference and Interface Files

```
let double (x : int) : int = x + x

let is_whitespace (c : char) =
  List.mem c [' '; '\n'; '\t'; '\r']

let version = 225

exception MyException
```

foo.ml

```
val double : int -> int

val is_whitespace : char -> bool

val version : int

exception MyException
```

foo.mli

In most cases, OCaml infers the signature of a given module (the annotation is optional)

In fact, we've been defining modules the entire time: *every file defines a module, whose name is the same as the filename (capitalized)*

Signature Inference and Interface Files

```
let double (x : int) : int = x + x

let is_whitespace (c : char) =
  List.mem c [' '; '\n'; '\t'; '\r']

let version = 225

exception MyException
```

foo.ml

```
val double : int -> int

val is_whitespace : char -> bool

val version : int

exception MyException
```

foo.mli

In most cases, OCaml infers the signature of a given module (the annotation is optional)

In fact, we've been defining modules the entire time: *every file defines a module, whose name is the same as the filename (capitalized)*

We can make signatures of files explicit with **.mli** files

Working with Modules

```
module type F00 =  
  sig  
    val double : int -> int  
    val is_whitespace : char -> bool  
    val version : int  
    exception MyException  
  end
```

```
let check c =  
  if Foo.version > 300 && Foo.is_whitespace c  
  then "okay"  
  else "not okay"
```

Once a module is defined, we can use values defined therein by **dot notation**

(This should feel somewhat familiar, again, we've been working with modules this whole time)

Opening Modules

```
open Foo
```

```
let check c =  
  if version > 300 && is_whitespace c  
  then "okay"  
  else "not okay"
```

Opening Modules

```
open Foo
```

```
let check c =  
  if version > 300 && is_whitespace c  
  then "okay"  
  else "not okay"
```

We can bring all definitions in a module into scope with the **open** keyword

Opening Modules

```
open Foo
```

```
let check c =  
  if version > 300 && is_whitespace c  
  then "okay"  
  else "not okay"
```

We can bring all definitions in a module into scope with the **open** keyword

Caution: Do this sparingly, it's like **import *** except worse because there's no overloading in OCaml

Opening Modules

```
open Foo
```

```
let check c =  
  if version > 300 && is_whitespace c  
  then "okay"  
  else "not okay"
```

We can bring all definitions in a module into scope with the **open** keyword

Caution: Do this sparingly, it's like **import *** except worse because there's no overloading in OCaml

If there are multiple definition of the function, the most recent open prevails

Abstract Types

```
module type S = sig
  type t
  type 'a t_param
  val op : t -> t -> t
  val op_param : 'a t_param -> 'a t_param -> 'a t_param
end
```

We can also define **abstract types** in modules

It allows us to define structures which are
type agnostic

example
(linalg.ml)

Results

Recall: Results

```
type ('a, 'b) result =  
  | Ok of 'a  
  | Error of 'b
```

A **result** is the same as an option, except that we carry data in the "None" case

Results play the same role as options, except they may help with things like error messages

Example

```
type error = LessThanOne | MoreThanOne
```

```
let exactly_one (f : 'a -> bool) (l : 'a list) : ('a, error) result =  
...
```

Example

```
type error = LessThanOne | MoreThanOne
```

```
let exactly_one (f : 'a -> bool) (l : 'a list) : ('a, error) result =  
...
```

Suppose we want to find the *only* element in a list that satisfies a given property

Example

```
type error = LessThanOne | MoreThanOne

let exactly_one (f : 'a -> bool) (l : 'a list) : ('a, error) result =
  ...
```

Suppose we want to find the *only* element in a list that satisfies a given property

But we also want to know why we failed...

Example

```
type error = LessThanOne | MoreThanOne

let exactly_one (f : 'a -> bool) (l : 'a list) : ('a, error) result =
...
```

Suppose we want to find the *only* element in a list that satisfies a given property

But we also want to know why we failed...

Results allow use to return more information in the failed case

Aside: Option Chaining

```
let user = {}; // user has no address  
alert( user?.address?.street ); // undefined (no error)
```

Aside: Option Chaining

```
let user = {}; // user has no address  
alert( user?.address?.street ); // undefined (no error)
```

Option chaining is a feature used in languages like javascript for *sequencing* functions on a possibly nonexistent object without having to explicitly check

Aside: Option Chaining

```
let user = {}; // user has no address  
alert( user?.address?.street ); // undefined (no error)
```

Option chaining is a feature used in languages like javascript for *sequencing* functions on a possibly nonexistent object without having to explicitly check

This is a pattern based on **monads** (taken from FP)

Advanced: let* Syntax

```
let foo =  
  let ( let* ) = Option.bind in  
  let* x = ox in  
  let* y = oy in  
  let* z = oz in  
  Some (x + y + z)
```

```
let bar =  
  bind ox (fun x ->  
    bind oy (fun y ->  
      bind oz (fun z ->  
        Some (x + y + z))))
```

```
let baz =  
  match ox with  
  | None -> None  
  | Some x -> (  
    match oy with  
    | None -> None  
    | Some y -> (  
      match oz with  
      | None -> None  
      | Some z -> Some (x + y + z)  
    )  
  )
```

let* syntax allows use to shorthand the use of the **bind** function, which allows us to avoid sequences of (annoying) pattern matches

Again, this is exactly option chaining

Disclaimer: It's never
required that you use monads

Workshop: linear algebra